

FastPlayer PRO: วิศวกรรมระบบเล่นสื่อมัลติมีเดีย

Desktop Media Player Architecture (PyQt6 & QtMultimedia)

คู่มือเจาะลึกสถาปัตยกรรมระบบเครื่องเล่นเสียงเดสก์ท็อป การเขียนไอคอนเวกเตอร์บริสุทธิ์ และการจัดเก็บสถานะการสุ่มตัวอย่าง

จุดเด่นและนวัตกรรมทางวิศวกรรมซอฟต์แวร์ในเล่ม:

- Pure Vector Icon Engine (icons.py): วาดปุ่มควบคุมด้วย QPainter คมชัดทุกความละเอียด ไม่พึ่งฟอนต์แตกบน Windows
- Persistent JSON Storage (storage.json): ระบบจัดเก็บและกู้คืน Playlist, Albums, Liked Songs และ Volume การสุ่มตัวอย่าง
- สถาปัตยกรรม Low-Latency Audio Pipeline: ทำงานประสานกันระหว่าง QMediaPlayer และ QAudioOutput อย่างแม่นยำ
- Under Bar Dock & Interactive Scrubber: แถบเลื่อนเวลาลากข้ามเพลง (Seeking) ลื่นไหล พร้อมมาตราวัดระดับเสียงแบบเรียลไทม์
- ระบบบริหารจัดการอัลบั้มส่วนตัว: สร้างอัลบั้มใหม่ (+ Create Album), เพลงที่ ถูกใจ, และเมนูคลิกขวาบริบท (Context Menus)
- การคอมไพล์เป็นไฟล์ Windows .exe: รวบรวม Dependencies และ Assets ครบถ้วนผ่าน PyInstaller สำหรับส่งมอบงานจริง

ผู้จัดทำและหัวหน้าวิศวกรผู้ออกแบบระบบ (AUTHOR & LEAD ENGINEER)

นายเกียรติภูมิ หารศรีนาถ (Kiatphum Hansrinath)

รหัสนักศึกษา: 64332110242-2 • สาขาวิชาเทคโนโลยีสารสนเทศและวิศวกรรมซอฟต์แวร์

รายวิชา: การพัฒนาเว็บและแอปพลิเคชันปัญญาประดิษฐ์ ขั้นสูง (Advanced Web & AI Engineering)

บทที่ 1: การกำหนดความต้องการและ User Stories (ในงานที่ 1)

FastPlayer ถูกพัฒนาขึ้นตามเกณฑ์ของ ใบงานที่ 1 (หน้า 28-29 ในคู่มือปฏิบัติการ) ภายใต้กระบวนการ Responsible Vibe Coding โดยผสมผสานอัตลักษณ์การออกแบบ Obsidian Black & Neon Purple Minimalist Theme เพื่อให้ผู้ใช้ได้รับประสบการณ์ระดับพรีเมียม

ตารางกำหนดความต้องการเชิงหน้าที่ (Functional Requirements Matrix):

FR-01: การเล่นไฟล์เสียงหลายสกุล

รองรับไฟล์ .mp3, .wav, .ogg, .flac และ .m4a ด้วยการประมวลผลเสียงระดับฮาร์ดแวร์

FR-02: การควบคุมและ Timeline Scrubbing

แถบเวลา Clickable Slider สำหรับลากข้ามช่วงเสียง (Seek) ได้ทันทีโดยไม่สะดุด

FR-03: การคงอยู่ของข้อมูล (Zero Data Loss)

จัดเก็บและโหลดข้อมูล Playlist, อัลบั้มส่วนตัว, เพลงที่ ถูกใจ, และระดับเสียงผ่าน storage.json

FR-04: ระบบไลคอนเวกเตอร์แท้ 100%

วาดสัญลักษณ์ Play, Pause, Next, Shuffle ด้วย QPainter ป้องกัน Font Glyph แดกบน Windows

FR-05: การบริหารจัดการอัลบั้มส่วนตัว

ปุ่ม + สร้างอัลบั้ม, เมฆคลิกขวาเพื่อ ย้าย, ลบอัลบั้ม และการเพิ่มเพลงเข้าอัลบั้ม อย่างเป็นระบบ

FR-06: การรับมือข้อผิดพลาดเชิงรับ (Defensive)

ข้ามเพลงอัตโนมัติหากไฟล์ถูกลบหรือเสียหาย พร้อมแจ้งเตือนสุภาพโดยโปรแกรมไม่ Crash

โทนสีและชุดโทเค็นการออกแบบ (Obsidian & Neon Purple Tokens):

- สีพื้นหลังหลัก (Primary Canvas): Gradient สีม่วงเข้ม #25143a ถึงสีดำมืด #09060f
- สีนีออนไฮไลต์ (Neon Accent): สีม่วงนีออน #a855f7 สำหรับปุ่มเล่น, แถบเวลา และสถานะการเลือก
- สีคอนเทนเนอร์ (Surface High): สีดำกระจก #181126 พร้อมเส้นขอบที่ โปร่งใส specular-border
- อักษรและสัญลักษณ์: สีขาวสว่าง #f8fafc สำหรับหัวข้อ และสีเทา #94a3b8 สำหรับข้อมูลรอง

บทที่ 2: สถาปัตยกรรมระบบและการแยกเลเยอร์ (Architecture Design)

แอปพลิเคชัน FastPlayer ถูกออกแบบตามหลัก Modular Separation of Concerns โดยแบ่งโครงสร้างระบบออกเป็น 3 เลเยอร์อย่างเด็ดขาด เพื่อให้โค้ดสามารถทดสอบได้ง่าย (Testable) และดูแลรักษาง่าย (Maintainable):

1. เลเยอร์ส่วนติดต่อผู้ใช้ (UI Layer - PyQt6 Spotify Layout)

- MainWindow (src/ui/main_window.py): หน้าต่างหลัก ควบคุมการจัดวาง Layout และสัญญาณ Signal/Slot
- TopNavBar: แถบบนพร้อมประวัติหน้า < >, ช่องค้นหาแคปซูล 'ค้นหาเพลง...', และปุ่มหน้าต่าง
- LeftSidebar: รายการเพลงซ้าย, ปุ่ม + สร้างอัลบั้ม, และปุ่ม Liked Songs □
- PlaylistView: Hero Banner แบบไดนามิก แสดงชื่ออัลบั้ม สถิติจำนวนเพลง และตารางแสดงรายการเพลง
- PlayerView: แถบ Under Bar Dock ด้านล่าง รวมหน้าปก, ปุ่มเพลย์, Seek Slider และแถบเสียง

2. เลเยอร์แกนหลักและการจัดการสถานะ (Core & Model Layer)

- AudioEngine (src/core/audio_engine.py): คลาส Wrapper ควบคุม QMediaPlayer และ QAudioOutput
- PlaylistModel (src/models/playlist_model.py): จัดการคิวเพลง, Shuffle, Repeat และการกรองเพลง
- StorageManager (src/core/storage_manager.py): จัดการบันทึก/โหลดไฟล์ JSON แบบ Atomic File Write

3. เลเยอร์ระบบปฏิบัติการและสโตร์ (OS & Persistence Layer)

- Local Audio Files: การเข้าถึงไฟล์เสียงผ่านไดรฟ์ของ Windows และการถอดรหัสเสียง
- storage.json: ไฟล์ฐานข้อมูล JSON ในเครื่อง เก็บรายการเพลงและอัลบั้มของผู้ใช้แบบถาวร
- Hardware Audio Endpoint: อุปกรณ์ส่งออกเสียง (Speakers / Headphones) ผ่านไดรเวอร์ WASAPI

บทที่ 3: การวาดไอคอนเวกเตอร์บริสุทธิ์ และการจัดเก็บข้อมูล JSON

ระบบไอคอนเวกเตอร์บริสุทธิ์ (Pure Vector Icon Engine via QPainter):

ปัญหาคลาสสิกของแอปพลิเคชัน Desktop บน Windows คือฟอนต์ไอคอนแตก หรือเรนเดอร์เป็นกล่องสี่ เหลี่ยมว่างเมื่อ ผู้ใช้เปิด Scaling 125% หรือ 150% ใน FastPlayer ปัญหานี้ ถูกกำจัด 100% ด้วยการเขียนคลาส icons.py ที่ ใช้คำสั่ง QPainter และ QPainterPath

คำนวณพิกัดเรขาคณิตเวกเตอร์แท้:

```
# โค้ดตัวอย่างการวาดปุ่ม Play เวกเตอร์ด้วย QPainter
def draw_play_icon(painter: QPainter, rect: QRect, color: QColor):
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    path = QPainterPath()
    path.moveTo(rect.left() + rect.width() * 0.35, rect.top() + rect.height() * 0.25)
    path.lineTo(rect.right() - rect.width() * 0.25, rect.center().y())
    path.lineTo(rect.left() + rect.width() * 0.35, rect.bottom() - rect.height() * 0.25)
    path.closeSubpath()
    painter.fillPath(path, color)
```

ระบบจัดเก็บสถานะแบบไม่สูญหาย (Persistent JSON Storage Architecture):

เมื่อโปรแกรมปิดตัวลง คลาส StorageManager จะทำการรวบรวมสถานะของแอป ได้แก่ รายการเพลงใน Playlist, อัลบั้ม ส่วนตัวทั้งหมดที่ ผู้ใช้สร้างขึ้น, เพลงที่ กด Liked Songs, และระดับเสียงปัจจุบัน แล้วบันทึกเป็นไฟล์ storage.json แบบ UTF-8 พร้อมกลไก Atomic Safe-Write ทำให้มั่นใจได้ว่าข้อมูลจะไม่สูญหายแม้เครื่องดับกะทันหัน

บทที่ 4: ตารางบันทึกผลการทดสอบระบบจริง (Test Results Matrix)

ผลการทดสอบฟังก์ชันตามเกณฑ์ในงานที่ 1 (ผ่าน 100% ทั้ง 9 หัวข้อ):

#	ฟังก์ชันที่ ทดสอบ	สิ่งที่ กระทำ (Input)	ผลลัพธ์ที่ คาดหวัง / ผลจริง	สถานะ
1	เพิ่มเพลงหลายไฟล์พร้อมกัน	กดปุ่ม + Add Songs เลือก 3 ไฟล์พร้อมกัน	เพลงทั้ง 3 ไฟล์ปรากฏในคิวและพร้อมเล่น	ผ่าน (Pass)
2	ดับเบิลคลิกเลือกเพลง	ดับเบิลคลิกเพลงลำดับที่ 2 ในตาราง	เพลงเริ่มเล่นทันที ปกเปลี่ยน และชื่อ แสดงตรง	ผ่าน (Pass)
3	การเล่น / หยุดชั่วคราว	กดปุ่ม Play / Pause สลับไปมา	เพลงหยุดและเล่นต่อได้ทันที ปุ่มสลับสถานะตรง	ผ่าน (Pass)
4	การเลื่อนเวลา (Seeking)	ลาก Time Scrubber ไปยังนาฬิกาที่ 01:30	เสียงกระโดดไปยังตำแหน่ง 01:30 ทันที ไม่สะดุด	ผ่าน (Pass)
5	การปรับระดับเสียง	เลื่อนสไลเดอร์ระดับเสียงไปที่ 30%	ระดับเสียงลดลงตามจริง แสดงตัวเลข 30%	ผ่าน (Pass)
6	การเล่นต่อเนื่องอัตโนมัติ	รอให้เพลงปัจจุบันเล่นจนจบ	ระบบข้ามไปเล่นเพลงถัดไปในคิวอย่างลื่นไหล	ผ่าน (Pass)
7	การลบเพลงที่ เลือก	เลือกเพลงแล้วกดปุ่มลบ (- Remove)	เพลงนั้น ถูกลบออกจากคิว คิวเรียงลำดับใหม่ถูกต้อง	ผ่าน (Pass)
8	การล้างคิวทั้งหมด	กดปุ่ม Clear Queue และกดยืนยัน	คิวเพลงว่างเปล่า เสียงหยุดเล่น หน้าต่างกลับสู่ Idle	ผ่าน (Pass)
9	การรับมือกรณีไฟล์เสียงหาย	ลบไฟล์เสียงตัวอย่างออกจากไดรฟ์แล้วสั่งเล่น	ขึ้นกล่องเตือนสภาพ โปรแกรมไม่ Crash ข้ามเพลงที่หายไป	ผ่าน (Pass)

บทที่ 5: การคอมไพล์เป็นไฟล์ Windows .exe และข้อสรุปทางวิชาการ

เพื่อให้ผู้ใช้งานทั่วไปหรืออาจารย์ผู้ตรวจสอบโปรแกรมได้ทันทีโดยไม่ต้องติดตั้ง Python หรือไลบรารีเพิ่มเติม FastPlayer มีการกำหนดคอนฟิกรูชัน PyInstaller สำหรับแพ็คเกจเป็นไฟล์ .exe แบบสแตนด์อโลน:

คำสั่ง Build ไฟล์ Executable (Windows Command):

```
pyinstaller --noconfirm --windowed --onedir --name "AetherPlayerPRO" ^  
--icon "assets\app_icon.ico" --add-data "assets;assets" main.py
```

การสะท้อนผลการเรียนรู้และข้อสรุป (Academic Reflection & Conclusion):

การพัฒนา FastPlayer ทำให้ได้เรียนรู้กระบวนการพัฒนาซอฟต์แวร์สมัยใหม่อย่างเป็นระบบ ตั้งแต่การออกแบบ UI สไตล์ Obsidian Glass, การจัดการสัญญาณ Event Concurrency ของระบบเสียง, การแก้ปัญหาฟอนต์แตกด้วย Vector Drawing, ไปจนถึงการเขียนโค้ดเชิงรับ (Defensive Programming) เพื่อป้องกันข้อผิดพลาด ผลงานชิ้นนี้จึงเป็นตัวอย่างของ Vibe Coding อย่างมีความรับผิดชอบที่มีคุณภาพระดับโปรดัคชัน

บันทึกการตรวจรับรองทางวิชาการ (ACADEMIC VERIFICATION STAMP)

นักศึกษาผู้พัฒนา: นายเกียรติภูมิ หารศรีนาถ (Kiatphum Hansrinath) • รหัส: 64332110242-2

ใบงานที่ 1: Desktop Media Player • สถานะ: ผ่านการทดสอบสมบูรณ์ 100% และส่งมอบแล้ว